

Local AI Stack for Pentesting Agent



To achieve a high "novelty factor" for an IEEE conference paper, you should lean into the **Self-Healing Loop** and the **Graph-Based Reasoning** aspects of your architecture. Most autonomous pentesting research focuses on simple LLM command generation; your innovation lies in **deterministic safety guards** and **long-term memory hygiene**.

Here is a structured development roadmap divided into four research-focused sprints.

Phase 1: Infrastructure & Data Grounding (Sprint 1)

Goal: Establish the "Memory" and "Intelligence" pipelines.

- **Dockerize Infrastructure:** Deploy Neo4j (Graph), ChromaDB (Vector), and n8n (Automation) using the provided architecture.
 - **n8n Intelligence Pipeline:** Configure n8n to fetch NVD feeds and push them to your `/update_intel` endpoint to populate ChromaDB.
 - **Graph Schema Setup:** Initialize the Neo4j schema following the host-port-vuln relationship to prevent data redundancy.
 - **Innovation Trigger:** Implement a "De-duplication Logic" in the n8n pipeline to ensure only unique exploit vectors are stored, which improves RAG (Retrieval-Augmented Generation) performance.
-

Phase 2: The Neuro-Symbolic Guard (Sprint 2)

Goal: Build the "Safety Layer" that prevents LLM hallucinations—a key novelty for research.

- **Pydantic Implementation:** Develop `src/agent/guard.py` using the `AttackCommand` model.
 - **Subnet Validation:** Implement the `validate_scope` function to ensure the agent never targets IPs outside your allowed range.
 - **Command Sanitization:** Build the `validate_safety` logic to strip dangerous flags like `-T5` or destructive commands.
 - **Novelty Factor:** Use "Negative Constraints" in your research paper to show how your agent is safer than "unbounded" LLM agents.
-

Phase 3: Self-Healing & Circuit Breaker (Sprint 3)

Goal: Implement the "Action" layer with error recovery.

- **Subprocess Runner:** Create `src/execution/runner.py` to handle tool execution with strict timeouts.
 - **Self-Healing Logic:** Design the feedback loop where `stderr` (errors) is sent back to the Planner for script rewriting.
 - **Circuit Breaker:** Implement the logic to abort a task if it fails more than twice, preventing infinite loops.
 - **Innovation Trigger:** Measure the "Success-to-Retry" ratio. This data is excellent for IEEE results sections to prove your agent's autonomy.
-

Phase 4: LangGraph Integration & Analysis (Sprint 4)

Goal: Orchestrate the nodes into a Directed Cyclic Graph (DCG).

- **State Management:** Define the `AgentState` object to track message history and retry counts.
- **Graph Logic:** Connect the Planner → Guard → Executor → Observer nodes using LangGraph.
- **Graph Hygiene:** Ensure the Observer node updates Neo4j only with "Open" ports to prevent "Graph Explosion".
- **Research Integration:** Run the agent against a known vulnerable lab (e.g., HackTheBox) and export the Neo4j graph as a "Visual Attack Map" for your paper.

Suggested Research Innovations for your Paper:

1. **Safety-First Autonomy:** Focus on the "Neuro-Symbolic" approach where LLM creativity is bound by Pydantic's deterministic rules.
2. **Stateful Pentesting:** Contrast your "Graph Memory" (Neo4j) against "Stateless" tools that forget context between steps.
3. **Autonomous Recovery:** Detail the "Circuit Breaker" as a mechanism to handle the inherent unreliability of LLM-generated code.

